```
»     language out there.  If you need to get working programs
»     delivered fast, you should add Ruby to your toolbox.
»   price:       49.50
»   image_url:   ruby.jpg
```

Note that the images referenced in `image_url` do need to exist for the tests to succeed. It doesn't matter what they are as long as they're in `app/assets/images` when the tests run. You can either create some yourself or use the ones provided in the downloadable code.

Now that we have a fixture file, we want Rails to load the test data into the `products` table when we run the unit test. And, in fact, Rails is already doing this (convention over configuration for the win!), but you can control which fixtures to load by specifying the following line in `test/models/product_test.rb`:

```
  class ProductTest < ActiveSupport::TestCase
»   fixtures :products
    #...
  end
```

The `fixtures` directive loads the fixture data corresponding to the given model name into the corresponding database table before each test method in the test case is run. The name of the fixture file determines the table that's loaded, so using `:products` will cause the `products.yml` fixture file to be used.

Let's say that again another way. In the case of our `ProductTest` class, adding the `fixtures` directive means that the `products` table will be emptied out and then populated with the three rows defined in the fixture before each test method is run.

Note that most of the scaffolding that Rails generates doesn't contain calls to the `fixtures` method. That's because the default for tests is to load *all* fixtures before running the test. Because that default is generally the one you want, there usually isn't any need to change it. Once again, conventions are used to eliminate the need for unnecessary configuration.

So far, we've been doing all our work in the development database. Now that we're running tests, though, Rails needs to use a test database. If you look in the